Лабораторная работа №2: приложение с UI для генератора билетовЛабораторная работа №2: приложение с UI для генератора билетов

Тип приложения: десктопное с графическим интерфейсом

Хранилище (вход): Excel — список студентов, Word — список билетов и вопросов

Хранилище (выход): Excel — журнал результатов

Варианты GUI-стека

Язык	Библиотека
Python	Tkinter (минимум зависимостей) / PyQt / customtkinter
C#	WPF / WinForms
Java	JavaFX / Swing
Node.js	Electron
Go	Fyne / Wails

1. Входные данные

`students.xlsx` — один лист на группу, имя листа = имя группы. Фамилия и имя студента — по одной строке на человека. Заголовок в строке 1, данные с строки 2: колонка A — Фамилия, колонка B — Имя.

`tickets.docx` — список билетов. Формат для парсинга:

```
Билет 1
1. Вопрос первый
2. Вопрос второй
3. Вопрос третий

Билет 2
...
```

Заголовок билета — строка вида `Билет N`, вопросы — нумерованные абзацы `N. текст`.

2. Логика интерфейса

1. При запуске первый выпадающий список заполняется именами листов `students.xlsx` (группы).
2. Кнопка «Сгенерировать билет» неактивна до выбора и группы, и студента.
3. После выбора группы второй выпадающий список заполняется студентами этой группы в формате «Фамилия Имя».
4. После выбора студента кнопка становится активной.

5. Клик по кнопке открывает окно результата: «Ваш билет № N» и три вопроса билета.
6. Результат дописывается в `results.xlsx`.
7. `ESC` закрывает окно результата и возвращает к выбору — приложение не завершается, готово к следующему студенту.

3. Назначение билета

Если студент (группа + Фамилия + Имя) уже есть в `results.xlsx` — берётся номер билета из **самой первой** записи по этому студенту, новый билет не генерируется. В журнал добавляется новая строка с этим же номером и пометкой «да» в колонке «Повтор».

Если студента в журнале ещё нет — билет выбирается случайно в диапазоне `1..N`, где N — количество билетов в `tickets.docx`. Новая строка пишется с пометкой «нет» в колонке «Повтор».

4. Формат `results.xlsx`

Создаётся при первом запуске, если отсутствует. Данные дописываются в конец, существующие строки не изменяются и не перезаписываются.

A	B	C	D	E	F
Группа	Фамилия	Имя	Номер билета	Дата и время	Повтор (да/нет)

5. Обязательные требования

- **Валидация источников.** Отсутствие `students.xlsx` или `tickets.docx` при запуске — понятное сообщение, приложение не падает.
- **Пустая группа.** Если у выбранной группы нет студентов — второй список пуст, кнопка остаётся неактивной, без исключения.
- **Блокировка файла.** Если `results.xlsx` открыт в Excel и недоступен для записи — сообщение, повтор попытки, без падения.
- **Устойчивость к некорректным билетам.** Если в `tickets.docx` меньше 3 вопросов у какого-то билета или заголовок не распознан — билет пропускается при парсинге, ошибка логируется, приложение продолжает работать с оставшимися билетами.
- **Сохранение сразу после генерации**, не при закрытии приложения.

6. Порядок выполнения

Окружение уже настроено и синхронизировано после Лаб 1 (GitHub, Git Bash, VS Code, Copilot Chat, авторизация) — шаги установки не повторяются.

Шаг 1 — Prompt Engineering. Составить детальное ТЗ на английском во внешней LLM (ChatGPT / Gemini / Copilot) для передачи в Copilot Chat — экономия токенов, точность генерации.

Шаг 2 — Активная разработка. Генерация UI, логики выпадающих списков, парсинга Excel/Word через Copilot: `Ctrl+I` для инлайн-правок, панель чата для структурных решений.

Шаг 3 — Unit-тесты. Команда `/tests` в Copilot Chat — минимум на: парсинг билета из Word, поиск студента по группе, идемпотентность (повторный выбор → тот же билет).

Шаг 4 — Контроль версий.

```
git checkout -b lab2-ui-tickets
git add .
git commit -m "Lab 2: UI-генератор билетов"
git push
```

7. Критерии приёмки

1. Выбор группы → список студентов соответствует листу.
2. Кнопка активна только при обоих выборах.
3. Билет и три вопроса отображаются корректно для случайного билета из файла.
4. Повторный выбор того же студента возвращает тот же номер билета.
5. `ESC` закрывает окно результата, не приложение.
6. Строка в `results.xlsx` появляется сразу после генерации, старые строки не тронуты.
7. Отсутствующий или заблокированный файл не роняет приложение.
8. Повторный выбор одного и того же студента добавляет новую строку в конце журнала, а не изменяет существующую; номер билета в новой строке совпадает с первой записью по этому студенту; колонка «Повтор» = «да».
9. По журналу можно восстановить полную историю всех генераций, включая повторы, в хронологическом порядке.